02-2、接口自动化测试Agent开发实战(一)

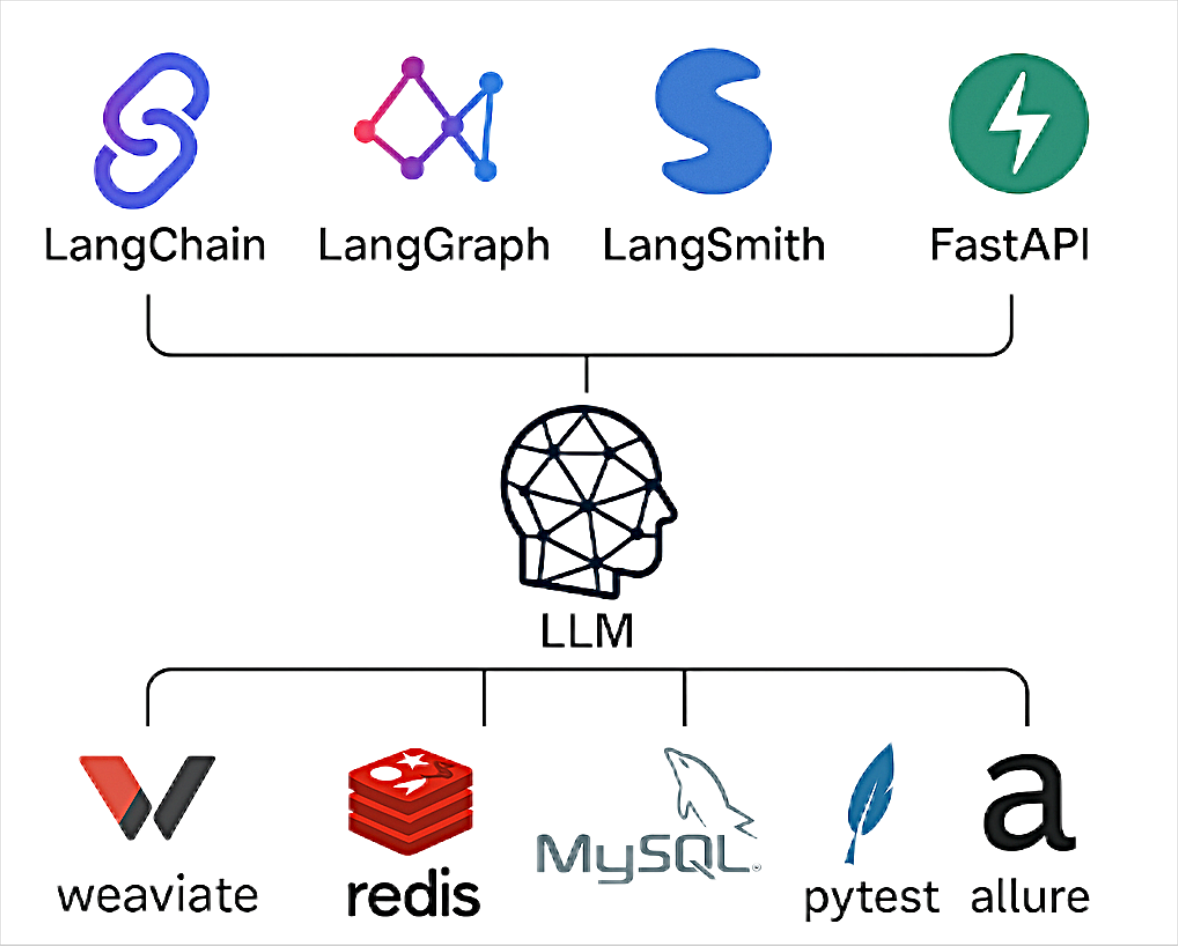
一、需求分析和系统设计

1、Agent的功能需求

> 需求一、根据接口文档-------> 生成单接口测试用例------>自动执行----->报告分析
>
> 需求二、根据业务描述-------> 生成业务流接口测试用例------>自动执行----->报告分析
>
> 需求三、根据功能用例-------> 转换为接口用例 ----->自动执行----->报告分析

2、开发技术栈

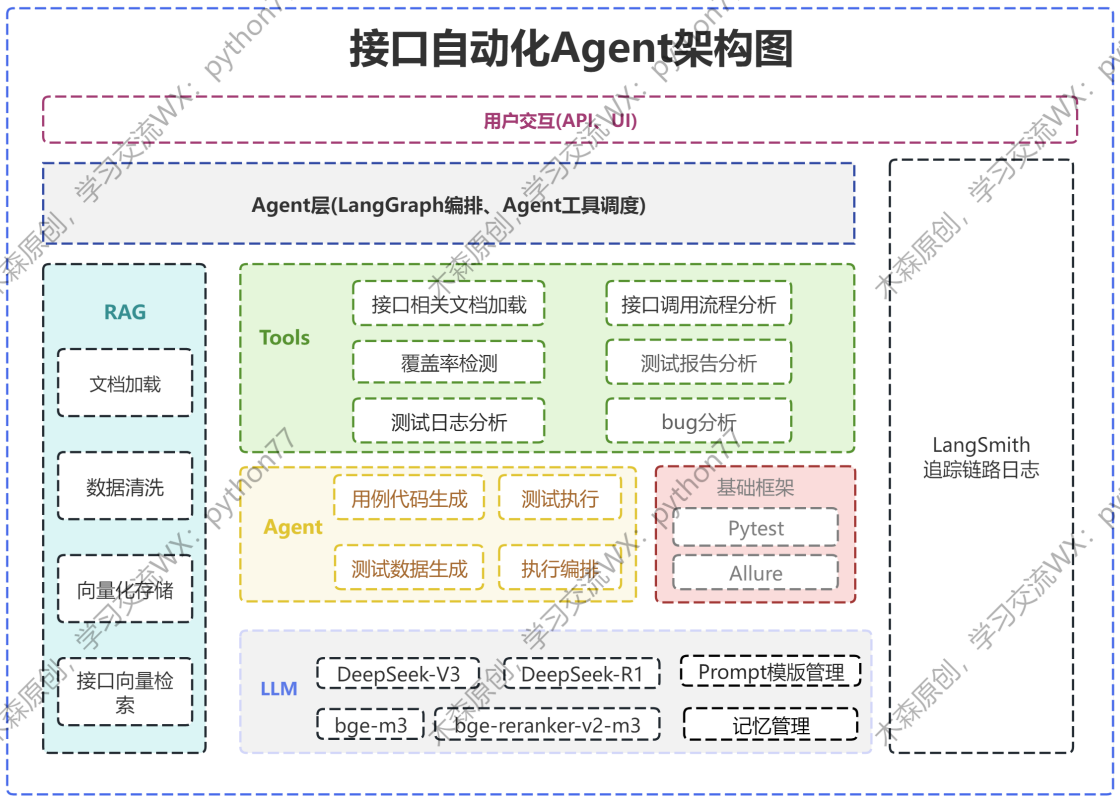
- **Langchin + langGraph + LangSmith + LLM(deepseek) + weaviate +FastApi+Redis+Mysql+Pytest+Allure**



-

3、Agent系统架构设计

- deepseek、QWQ



二、项目工程结构

核心框架（LangChain + LangGraph）

```
├── .env
├── main.py                    # 项目入口
├── configs/
│   └── settings.py            # 环境变量加载、全局配置（模型、路径等）
├── data/                      # 存放接口/需求文档（可 URL 或 JSON 文件）
│   ├── api_docs/
├── agents/                   # 主 Agent 构建逻辑（LangGraph）
│   ├── __init__.py
├── tools/                    # 封装的相关工具
│   ├── __init__.py
├── rag/                      # 向量存储+知识库检索
│   ├── __init__.py
├── prompts/                  # 提示词模版
│   ├── __init__.py
├── workflows/                # LangGraph步骤流程封装
│   ├── __init__.py
└── server/                   # FastAPI 接口（可部署为 Web 应用）
    └── __init__.py
```

三、项目RAG知识库搭建之接口数据清洗

如果要生成高可用的接口自动化的用例，必须得有详细的接口文档，字段约束说明，返回数据说明，以及相关的数据库表结构，

1、在线API文档解析

Yapi、Swagger 、openap3.0、apifox，飞书的文档、PDF、Word、MD,没有标准的接口文档(抓包的Curl请求包)……

```
# 将各种类型的接口文档------>进行数据清洗----------->得到统一的格式(提升RAG的召回率)------>向量数据库里面存储
```

1.2、Swagger 和 openapi文档区别

Swagger 和 OpenAPI 文档本质上是同一技术规范的不同发展阶段，但在实际使用中仍存在一些关键区别

维度	Swagger	OpenAPI
规范类型	旧称（2.0及之前版本）	新称（3.x版本）
维护方	SmartBear（原）	Linux基金会（现）
扩展性	有限	支持更灵活的扩展（`x-`字段）
文件后缀	常用`.swagger.json`	推荐`.openapi.json`

1.2、接口数据处理

- `openapi`在线接口文档地址示例：http://www.mstest.vip:8888/openapi.json
- swagger在线接口文档地址示例：https://petstore.swagger.io/v2/swagger.json

  OpenAPI 3.x特点:接口数据都是json格式,基本信息和请求参数，响应字段是分开的，

  ```
  # 文档基本信息
  print(data['info'])
  # 接口列表
  print(data['paths'])
  # 参数字段 和 响应字段
  print(data['components']['schemas'])
  ```

1.3、openapi解析脚本

```
import json
from typing import Dict, Any, List, Union


class OpenAPIAggregator:
    """
    OpenAPI 聚合器，用于加载、解析和聚合 OpenAPI 规范中的接口信息。

    参数:
    - openapi_path: OpenAPI 规范文件的路径。
    """
```

```
    def __init__(self, openapi_path: str):
        self.openapi_path = openapi_path
        self.openapi = self._load_openapi()
        self.components = self.openapi.get("components", {}).get("schemas", {})
        self.parameter_defs = self.openapi.get("components",
{}).get("parameters", {})
        self.request_body_defs = self.openapi.get("components",
{}).get("requestBodies", {})

    def _load_openapi(self) -> Dict[str, Any]:
        """
        加载 OpenAPI 规范文件。

        返回:
        - 解析后的 OpenAPI 规范字典。
        """
        with open(self.openapi_path, "r", encoding="utf-8") as f:
            return json.load(f)

    def _resolve_ref(self, ref: str) -> Any:
        """
        解析 $ref 引用。

        参数:
        - ref: 引用字符串，以 #/components 开头。

        返回:
        - 引用所指向的 schema、parameter 或 requestBody。
        """
        if ref.startswith("#/components/schemas/"):
            schema_name = ref.split("/")[-1]
            return self.components.get(schema_name, {"missing_schema":
schema_name})
        elif ref.startswith("#/components/parameters/"):
            param_name = ref.split("/")[-1]
            return self.parameter_defs.get(param_name, {"missing_param":
param_name})
        elif ref.startswith("#/components/requestBodies/"):
            body_name = ref.split("/")[-1]
            return self.request_body_defs.get(body_name,
{"missing_request_body": body_name})
        else:
            return {"unresolved_ref": ref}

    def _parse_schema(self, schema: Dict[str, Any]) -> Union[Dict[str, Any],
List[Any]]:
        """
        递归解析 schema。

        参数:
        - schema: 待解析的 schema 字典。

        返回:
        - 解析后的 schema。
        """
        if '$ref' in schema:
            schema = self._resolve_ref(schema['$ref'])
```

```
        if schema.get('type') == 'object':
            result = {}
            for prop, prop_info in schema.get('properties', {}).items():
                result[prop] = self._parse_schema(prop_info) if '$ref' in
prop_info else {
                    "type": prop_info.get("type"),
                    "description": prop_info.get("description", "")
                }
            return result
        elif schema.get('type') == 'array' and 'items' in schema:
            return [self._parse_schema(schema['items'])]
        else:
            return {
                "type": schema.get("type"),
                "description": schema.get("description", "")
            }

    def aggregate(self) -> List[Dict[str, Any]]:
        """
        聚合 OpenAPI 规范中的接口信息。

        返回:
        - 包含所有接口信息的列表。
        """
        paths = self.openapi.get("paths", {})
        api_list = []

        for path, methods in paths.items():
            for method, details in methods.items():
                api_entry = {
                    "path": path,
                    "method": method.upper(),
                    "summary": details.get("summary", ""),
                    "description": details.get("description", ""),
                    "parameters": [],
                    "requestBody": {},
                    "responses": {}
                }

                # parameters
                for param in details.get("parameters", []):
                    if "$ref" in param:
                        param = self._resolve_ref(param["$ref"])
                    schema = param.get("schema", {})
                    api_entry["parameters"].append({
                        "name": param.get("name"),
                        "in": param.get("in"),
                        "required": param.get("required", False),
                        "type": schema.get("type"),
                        "description": param.get("description", "")
                    })

                # request body
                req_body = details.get("requestBody", {})
                if "$ref" in req_body:
                    req_body = self._resolve_ref(req_body["$ref"])
                content = req_body.get("content", {})
                if "application/json" in content:
```

```
                    schema = content["application/json"].get("schema", {})
                    api_entry["requestBody"] = self._parse_schema(schema)

                # responses
                for status_code, resp in details.get("responses", {}).items():
                    content = resp.get("content", {})
                    if "application/json" in content:
                        schema = content["application/json"].get("schema", {})
                        api_entry["responses"][status_code] =
self._parse_schema(schema)

                api_list.append(api_entry)

        return api_list

    def save_to_json(self, output_path: str, data: List[Dict[str, Any]]):
        """
        将聚合后的接口信息保存为 JSON 文件。

        参数:
        - output_path: 输出文件的路径。
        - data: 聚合后的接口信息列表。
        """
        with open(output_path, "w", encoding="utf-8") as f:
            json.dump(data, f, ensure_ascii=False, indent=2)
        print(f"已保存聚合后的接口信息，共 {len(data)} 条接口 → {output_path}")


# 使用示例
if __name__ == "__main__":
    aggregator = OpenAPIAggregator("openapi.json")
    api_info = aggregator.aggregate()
    print(api_info)
```

1.4、Swagger文档处理

```
import json
from typing import Dict, List, Any


class SwaggerV2Parser:
    """适配 Swagger 2.0 接口文档，提取路径、方法、参数和响应。"""

    def __init__(self, swagger_path: str):
        with open(swagger_path, "r", encoding="utf-8") as f:
            self.spec = json.load(f)
        self.definitions = self.spec.get("definitions", {})

    def _resolve_ref(self, ref: str) -> Dict[str, Any]:
        """解析 $ref 引用。"""
        ref_name = ref.split("/")[-1]
        return self.definitions.get(ref_name, {"unresolved": ref})

    def _parse_schema(self, schema: Dict[str, Any]) -> Any:
        """递归解析 schema。"""
        if "$ref" in schema:
```

```
            schema = self._resolve_ref(schema["$ref"])
        if schema.get("type") == "object":
            return {k: self._parse_schema(v) for k, v in
schema.get("properties", {}).items()}
        elif schema.get("type") == "array":
            return [self._parse_schema(schema["items"])]
        return {
            "type": schema.get("type"),
            "description": schema.get("description", "")
        }

    def parse(self) -> List[Dict[str, Any]]:
        """解析所有接口。"""
        results = []
        for path, methods in self.spec.get("paths", {}).items():
            for method, info in methods.items():
                item = {
                    "path": path,
                    "method": method.upper(),
                    "summary": info.get("summary", ""),
                    "parameters": [],
                    "responses": {}
                }

                # 处理 parameters（请求参数）
                for param in info.get("parameters", []):
                    if "$ref" in param:
                        param = self._resolve_ref(param["$ref"])
                    item["parameters"].append({
                        "name": param.get("name"),
                        "in": param.get("in"),
                        "type": param.get("type"),
                        "required": param.get("required", False),
                        "description": param.get("description", "")
                    })

                # 处理 responses
                for status, resp in info.get("responses", {}).items():
                    schema = resp.get("schema", {})
                    item["responses"][status] = self._parse_schema(schema)

                results.append(item)
        return results


if __name__ == "__main__":
    parser = SwaggerV2Parser("swagger.json")
    api_list = parser.parse()
    print(json.dumps(api_list, ensure_ascii=False, indent=2))
```

2、文档数据清洗

为了把接口文档存储的向量数据库，提高检索的成功率，还需要对接口文档数据进行清洗，保留关键字段，压缩结构嵌套，增强可读性、可检索性、LLM 嵌入效果更佳。

```
    def format_api_entry(self, entry: Dict) -> str:
        lines = [
            f"接口地址：{entry['method']} {entry['path']}",
            f"接口说明：{entry.get('summary') or entry.get('description',
'')}".strip()
        ]

        if entry.get("parameters"):
            lines.append("请求参数：")
            for param in entry["parameters"]:
                lines.append(
                    f"- {param['name']}（{param.get('type', 'unknown')}，{'必填'
if param.get('required') else '选填'}）：{param.get('description', '')}"
                )

        if entry.get("requestBody"):
            lines.append("请求体示例：")
            lines.append(json.dumps(entry["requestBody"], ensure_ascii=False,
indent=2))

        if entry.get("responses"):
            lines.append("响应示例：")
            for code, resp in entry["responses"].items():
                lines.append(f"状态码 {code}：")
                lines.append(json.dumps(resp, ensure_ascii=False, indent=2))

        return "\n".join(lines)
```